

TECHNICAL DOCUMENTATION

AI-HRMS

Comprehensive developer reference for the AI-powered Human Resource Management System. Covers architecture, technology stack, database design, API endpoints, frontend structure, security model, AI integration, deployment, and contributing guidelines.

Version 1.0

May 2026

Z.AI

Table of Contents

1. Architecture Overview	5
1.1 System Architecture	5
1.2 Data Flow Architecture	5
1.3 Client-Side State Management	6
1.4 AI Integration Architecture	6
2. Technology Stack	6
2.1 Frontend Stack	6
2.2 Backend Stack	7
2.3 Development and Deployment	7
3. Project Structure	8
3.1 Directory Tree	8
3.2 Directory Descriptions	9
4. Database Design	10
4.1 Entity-Relationship Overview	10
4.2 Complete Schema Documentation	10
4.3 Relationships and Foreign Keys	14
4.4 Indexing Strategy	14
4.5 Migration Management	14
5. API Documentation	15

5.1 API Overview	15
5.2 API Endpoints Reference	15
5.3 Error Handling	17
5.4 Authentication (Planned)	18
5.5 Rate Limiting	18
6. Frontend Architecture	18
6.1 Component Hierarchy	18
6.2 Module Component Structure	18
6.3 State Management with Zustand	19
6.4 Data Flow Pattern	20
6.5 Routing and Navigation	20
6.6 Responsive Design	20
6.7 Theme System	20
7. Security Architecture	21
7.1 RBAC Implementation	21
7.2 Permission Model	21
7.3 Predefined Roles	22
7.4 Data Masking Strategy	22
7.5 Audit Logging	22
7.6 API Security	22
7.7 Input Validation	23
8. AI Integration	23

8.1 AI Chatbot Implementation	23
8.2 AI Attrition Prediction	24
8.3 AI Fit Score Calculation	24
8.4 AI Learning Path Recommendations	25
8.5 AI Review Question Generation	25
9. Development Workflow	25
9.1 Local Development Setup	25
9.2 Environment Configuration	26
9.3 Running the Development Server	26
9.4 Code Style and Linting	26
9.5 Git Workflow	26
9.6 Testing Strategy	26
10. Deployment Guide	27
10.1 Vercel Deployment	27
10.2 GitHub Actions CI/CD Pipeline	27
10.3 Environment Variable Management	28
10.4 Database Migration in Production	28
10.5 PostgreSQL Setup for Production	28
10.6 Monitoring and Logging	29
10.7 Error Tracking	29
11. Performance Optimization	29

11.1 Rendering Strategy	29
11.2 API Response Caching	29
11.3 Bundle Optimization	30
11.4 Image Optimization	30
11.5 Database Query Optimization	30
12. Contributing Guidelines	30
12.1 Code Style Guide	30
12.2 PR Process	31
12.3 Commit Message Conventions	31
12.4 Branch Strategy	32
13. Troubleshooting	32
13.1 Common Build Errors	32
13.2 Database Connection Issues	32
13.3 Deployment Issues	33
13.4 Performance Issues	33

1. Architecture Overview

1.1 System Architecture

AI-HRMS is a full-stack Next.js application built on the App Router paradigm. The system employs a monolithic architecture where the frontend and backend are co-located within a single Next.js project. The frontend renders React components on the server side where possible (RSC), while interactive modules use client-side rendering with Zustand for state management. The backend consists of API routes that interface with a Prisma ORM layer, abstracting database operations across SQLite (development) and PostgreSQL (production).

High-Level Component Diagram

The following table describes the major architectural components and their responsibilities:

Layer	Component	Technology	Responsibility
Presentation	React Components	React 19 + RSC	UI rendering, user interaction
State	Zustand Store	Zustand	Client-side state management
API	Next.js API Routes	App Router /api	RESTful endpoints, data orchestration
Data Access	Prisma ORM	Prisma Client	Database queries, migrations, seeding
Database	SQLite / PostgreSQL	Prisma Adapter	Persistent data storage
AI	z-ai-web-dev-sdk	SDK Client	AI chat, predictions, recommendations
Deployment	Vercel	Vercel CLI	CI/CD, edge functions, CDN

Table 1: High-Level Architectural Components

1.2 Data Flow Architecture

Data flows through the AI-HRMS system in a predictable, layered pattern. The user interacts with React components in the browser, which trigger state changes via Zustand actions. For data persistence, client components call API routes via `fetch()`, which process requests through Prisma ORM to the database. The response flows back through the same chain. For AI features, the API routes invoke the `z-ai-web-dev-sdk`, which communicates with the AI service backend.

Request Flow: User Action → React Component → Zustand Store → `fetch()` → API Route → Prisma Client → Database

Response Flow: Database → Prisma Client → API Route → JSON Response → Zustand Store Update → React Re-render

AI Flow: User Query → API Route (/api/hr/chat) → z-ai-web-dev-sdk → AI Service → Streamed Response → Client Display

1.3 Client-Side State Management

AI-HRMS uses Zustand as its primary client-side state management library. Zustand was chosen over Redux for its minimal boilerplate, TypeScript-first design, and excellent performance characteristics. The store follows a modular pattern where each HR module (employees, attendance, leave, payroll, etc.) has its own slice of state, but all slices are combined into a single store for simplicity.

Store Shape (simplified):

```
interface HRStore {
  // Module states
  employees: Employee[];
  departments: Department[];
  attendance: AttendanceRecord[];
  leaves: LeaveRequest[];
  payroll: PayrollRecord[];
  // ... 9 module states
  // UI states
  activeModule: string;
  sidebarCollapsed: boolean;
  chatMessages: ChatMessage[];
  // Actions
  setActiveModule: (module: string) => void;
  toggleSidebar: () => void;
  addChatMessage: (msg: ChatMessage) => void;
}
```

1.4 AI Integration Architecture

AI-HRMS integrates with AI capabilities through the z-ai-web-dev-sdk. The integration follows a server-side proxy pattern where all AI requests are routed through Next.js API routes. This ensures API keys remain server-side and enables streaming responses to the client. The AI features are divided into four categories: conversational AI (chatbot), predictive analytics (attrition prediction), recommendation engine (learning paths), and automated assessment (review questions).

2. Technology Stack

2.1 Frontend Stack

Technology	Version	Purpose	Configuration
------------	---------	---------	---------------

Next.js	16	Full-stack React framework	App Router, SSR/CSR hybrid
React	19	UI component library	Concurrent features, RSC
TypeScript	5	Type-safe JavaScript	Strict mode enabled
Tailwind CSS	4	Utility-first CSS framework	JIT mode, custom theme
shadcn/ui	Latest	Component library (Radix UI)	Emerald/teal professional theme
Recharts	Latest	Charting library	Area, Bar, Pie, Radar charts
Zustand	Latest	State management	Single store, modular slices
Lucide React	Latest	Icon library	Tree-shakeable SVG icons

Table 2: Frontend Technology Stack

2.2 Backend Stack

Technology	Version	Purpose	Notes
Next.js API Routes	16	Server-side API endpoints	App Router /api directory
Prisma ORM	Latest	Database ORM	Schema-first, type-safe queries
SQLite	3	Development database	File-based, zero config
PostgreSQL	15+	Production database	Recommended for scale
z-ai-web-dev-sdk	Latest	AI service integration	Chat, predictions, recommendations

Table 3: Backend Technology Stack

2.3 Development and Deployment

Technology	Purpose	Notes
npm / bun	Package management	bun for speed, npm for compatibility
Vercel	Hosting and deployment	Zero-config Next.js deployment
GitHub Actions	CI/CD pipeline	Planned integration
ESLint	Code linting	Next.js config extends
Prettier	Code formatting	Consistent style enforcement

Table 4: Development and Deployment Stack

3. Project Structure

3.1 Directory Tree

The AI-HRMS project follows a feature-based organization within the Next.js App Router convention. Each module groups its components, and shared utilities are centralized in `/src/lib`. The complete directory structure is documented below:

```

ai-hrms/
|-- prisma/
|   |-- schema.prisma      # Database schema (20+ models)
|   |-- seed.ts            # Seed script with demo data
|   |-- migrations/        # Migration history
|-- public/
|   |-- images/            # Static image assets
|-- src/
|   |-- app/
|   |   |-- layout.tsx     # Root layout with sidebar
|   |   |-- page.tsx       # Dashboard (home page)
|   |   |-- globals.css    # Global styles + Tailwind
|   |   |-- api/
|   |       |-- hr/
|   |           |-- employees/    # Employee CRUD API
|   |           |-- departments/  # Department API
|   |           |-- attendance/   # Attendance API
|   |           |-- leave/        # Leave management API
|   |           |-- payroll/      # Payroll API
|   |           |-- expenses/     # Expense API
|   |           |-- performance/  # Performance API
|   |           |-- recruitment/  # Recruitment API
|   |           |-- training/     # Training API
|   |           |-- chat/         # AI chat API
|   |           |-- dashboard/    # Dashboard stats API
|   |           |-- reports/      # Reports API
|   |           |-- settings/     # Settings API
|   |-- components/
|   |   |-- hrms/
|   |       |-- EmployeeModule.tsx
|   |       |-- AttendanceModule.tsx
|   |       |-- LeaveModule.tsx
|   |       |-- PayrollModule.tsx
|   |       |-- ExpenseModule.tsx
|   |       |-- PerformanceModule.tsx
|   |       |-- RecruitmentModule.tsx
|   |       |-- TrainingModule.tsx
|   |       |-- DashboardModule.tsx
|   |   |-- ui/              # shadcn/ui components
|   |       |-- button.tsx
|   |       |-- card.tsx
|   |       |-- dialog.tsx
|   |       |-- table.tsx
|   |       |-- ... (30+ components)
|   |-- hooks/
|   |   |-- use-mobile.ts    # Mobile detection hook
|-- lib/
|   |-- utils.ts             # Utility functions (cn helper)
|   |-- data.ts              # Mock data for all modules
|   |-- store.ts             # Zustand store definition

```

3.2 Directory Descriptions

Directory	Purpose	Key Files
/src/app	Pages and API routes (App Router)	layout.tsx, page.tsx, /api/*
/src/components/hrms	Module components (9 modules)	*Module.tsx files
/src/components/ui	shadcn/ui primitives (Radix UI)	button, card, dialog, table, etc.
/src/lib	Utilities, data, store, db client	utils.ts, data.ts, store.ts, db.ts
/src/hooks	Custom React hooks	use-mobile.ts
/prisma	Schema, seed, migrations	schema.prisma, seed.ts
/public	Static assets	images/*

Table 5: Directory Descriptions

4. Database Design

4.1 Entity-Relationship Overview

The AI-HRMS database consists of 20+ models organized around the core Employee entity. Employee has relationships to Department, Role, Attendance, Leave, Payroll, Performance, Assets, Documents, Skills, and Course Enrollments. The design follows normalization principles up to 3NF, with deliberate denormalization in the Employee model for frequently accessed fields. Foreign keys are enforced at the Prisma level, and cascade delete is configured carefully to prevent accidental data loss.

4.2 Complete Schema Documentation

4.2.1 Employee

Field	Type	Constraints	Description
id	Int	PK, autoincrement	Unique employee identifier
employeeId	String	Unique, @default(uuid())	External-facing employee ID
firstName	String	Required	First name of the employee
lastName	String	Required	Last name of the employee
email	String	Unique, required	Work email address
phone	String?	Optional	Phone number
departmentId	Int	FK -> Department.id	Department assignment
roleId	Int	FK -> Role.id	Role assignment

position	String	Required	Job title / position
hireDate	DateTime	Required	Date of hire
status	EmployeeStatus	Default: ACTIVE	ACTIVE, ON_LEAVE, TERMINATED
salary	Float?	Optional	Base salary amount
avatar	String?	Optional	Avatar image URL
address	String?	Optional	Physical address
emergencyContact	String?	Optional	Emergency contact info
dateOfBirth	DateTime?	Optional	Date of birth
createdAt	DateTime	@default(now())	Record creation timestamp
updatedAt	DateTime	@updatedAt	Last update timestamp

Table 6: Employee Model Fields

4.2.2 Department

Field	Type	Constraints	Description
id	Int	PK, autoincrement	Unique department identifier
name	String	Unique, required	Department name
description	String?	Optional	Department description
headId	Int?	FK -> Employee.id	Department head reference
budget	Float?	Optional	Annual department budget
createdAt	DateTime	@default(now())	Record creation timestamp
updatedAt	DateTime	@updatedAt	Last update timestamp

Table 7: Department Model Fields

4.2.3 Role

Field	Type	Constraints	Description
id	Int	PK, autoincrement	Unique role identifier
name	String	Unique, required	Role name (e.g., "HR Manager")
description	String?	Optional	Role description

permissions	Json	Required	Permission matrix (module -> actions)
isDefault	Boolean	@default(false)	Default role for new users
createdAt	DateTime	@default(now())	Record creation timestamp

Table 8: Role Model Fields

4.2.4 Attendance

Field	Type	Constraints	Description
id	Int	PK, autoincrement	Unique attendance record ID
employeeId	Int	FK -> Employee.id, required	Employee reference
date	DateTime	Required	Attendance date
checkIn	DateTime?	Optional	Check-in timestamp
checkOut	DateTime?	Optional	Check-out timestamp
status	AttendanceStatus	Default: PRESENT	PRESENT, ABSENT, LATE, HALF_DAY
note	String?	Optional	Attendance note/reason

Table 9: Attendance Model Fields

4.2.5 Leave

Field	Type	Constraints	Description
id	Int	PK, autoincrement	Unique leave request ID
employeeId	Int	FK -> Employee.id, required	Employee reference
type	LeaveType	Required	ANNUAL, SICK, PERSONAL, MATERNITY, PATERNITY
startDate	DateTime	Required	Leave start date
endDate	DateTime	Required	Leave end date
status	LeaveStatus	Default: PENDING	PENDING, APPROVED, REJECTED
reason	String?	Optional	Reason for leave
approverId	Int?	FK -> Employee.id	Approving manager
createdAt	DateTime	@default(now())	Request creation timestamp

Table 10: Leave Model Fields

4.2.6 Payroll

Field	Type	Constraints	Description
id	Int	PK, autoincrement	Unique payroll record ID
employeeId	Int	FK -> Employee.id, required	Employee reference
month	String	Required	Payroll month (YYYY-MM format)
basicSalary	Float	Required	Base salary amount
allowances	Float	@default(0)	Total allowances
deductions	Float	@default(0)	Total deductions
tax	Float	@default(0)	Tax amount
netSalary	Float	Required	Net salary after deductions
status	PayrollStatus	Default: PENDING	PENDING, PROCESSED, PAID
paymentDate	DateTime?	Optional	Actual payment date

Table 11: Payroll Model Fields

4.2.7 Additional Models Summary

The remaining models follow similar patterns. Below is a summary of all additional models and their key fields:

Model	Key Fields	Relationships	Description
Expense	title, amount, category, date, status	FK -> Employee	Employee expense claims
Performance	rating, reviewDate, goals, feedback	FK -> Employee, Reviewer	Performance reviews
Asset	name, type, serialNumber, status	FK -> Employee	Company asset tracking
Document	name, type, fileUrl, category	FK -> Employee	Document management
Skill	name, category, description	Many-to-Many via EmployeeSkill	Skills catalog
EmployeeSkill	proficiency, certifiedOn	FK -> Employee, Skill	Employee-skill junction
Course	title, description, duration, category	Has CourseEnrollments	Training courses
CourseEnrollment	status, progress, completedAt	FK -> Employee, Course	Course enrollments

Job	title, department, type, status	Has Candidates	Job postings
Candidate	name, email, resumeUrl, stage	FK -> Job	Job candidates
AuditLog	action, entity, entityId, changes	FK -> Employee	Audit trail logging
ApprovalWorkflow	type, status, currentStep	FK -> Employee	Approval processes
CompanyPolicy	title, category, content	None	Company policy documents
Shift	name, startTime, endTime, type	FK -> Employee (many)	Work shift definitions
Holiday	name, date, type	None	Company holiday calendar

Table 12: Additional Models Summary

4.3 Relationships and Foreign Keys

The Employee model serves as the central entity with foreign key relationships to Department, Role, and all transaction models (Attendance, Leave, Payroll, Expense, Performance, Asset, Document). Department has a self-referential headId pointing back to Employee. The many-to-many relationship between Employee and Skill is resolved through the EmployeeSkill junction table, which also stores proficiency level and certification date.

Cascade Behavior: When a Department is deleted, Employee.departmentId is set to null (SetNull). When an Employee is deleted, all related records (Attendance, Leave, Payroll, etc.) are cascade-deleted. This is intentional to maintain data integrity. AuditLog records use ON DELETE CASCADE to the employee but store the employee name redundantly for historical reference.

4.4 Indexing Strategy

Prisma automatically creates indexes on primary keys and unique fields. Additional indexes are defined in the schema for frequently queried fields:

- Employee: @@index([departmentId]), @@index([status]), @@index([email])
- Attendance: @@index([employeeId, date]), @@index([date])
- Leave: @@index([employeeId, status]), @@index([startDate])
- Payroll: @@index([employeeId, month]), @@index([status])
- AuditLog: @@index([entity, entityId]), @@index([createdAt])

4.5 Migration Management

Database migrations are managed through Prisma Migrate. The workflow is as follows:

- **Development:** After modifying schema.prisma, run `npx prisma migrate dev --name description` to create and apply a migration.
- **Production:** Run `npx prisma migrate deploy` to apply pending migrations. Never use `migrate dev` in production.
- **Reset:** `npx prisma migrate reset` drops all data and re-applies migrations. Use only in development.
- **Seeding:** After migration, run `npx prisma db seed` to populate demo data via `prisma/seed.ts`.

5. API Documentation

5.1 API Overview

AI-HRMS exposes 14 RESTful API endpoints under the `/api/hr/` path prefix. All endpoints follow consistent patterns for request/response formatting, error handling, and pagination. The API is built on Next.js App Router route handlers and uses Prisma for data access. All endpoints return JSON responses with standardized error codes.

5.2 API Endpoints Reference

5.2.1 Employees API

Base Path: `/api/hr/employees`

Method	Path	Description	Auth Required
GET	<code>/api/hr/employees</code>	List all employees (paginated)	Yes (planned)
GET	<code>/api/hr/employees/[id]</code>	Get employee by ID	Yes (planned)
POST	<code>/api/hr/employees</code>	Create new employee	Yes (planned)
PUT	<code>/api/hr/employees/[id]</code>	Update employee	Yes (planned)
DELETE	<code>/api/hr/employees/[id]</code>	Delete employee	Yes (planned)

Table 13: Employees API Endpoints

GET `/api/hr/employees` - List Employees

Query Parameters:

- `page` (number, default: 1) - Page number for pagination
- `limit` (number, default: 20) - Items per page (max: 100)
- `department` (string, optional) - Filter by department name
- `status` (string, optional) - Filter by employee status
- `search` (string, optional) - Search by name or email

Response (200 OK):

```
{
  "data": [
    {
      "id": 1,
      "employeeId": "uuid-xxx",
      "firstName": "John",
      "lastName": "Doe",
      "email": "john.doe@company.com",
      "department": { "id": 1, "name": "Engineering" },
      "position": "Senior Developer",
      "status": "ACTIVE"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "totalPages": 8
  }
}
```

POST /api/hr/employees - Create Employee

Request Body:

```
{
  "firstName": "Jane",
  "lastName": "Smith",
  "email": "jane.smith@company.com",
  "departmentId": 1,
  "roleId": 2,
  "position": "Product Designer",
  "hireDate": "2026-05-01T00:00:00Z",
  "salary": 85000
}
```

5.2.2 All API Endpoints Summary

#	Endpoint	Methods	Description
1	/api/hr/employees	GET, POST	Employee CRUD operations
2	/api/hr/employees/[id]	GET, PUT, DELETE	Single employee operations
3	/api/hr/departments	GET, POST	Department management
4	/api/hr/attendance	GET, POST	Attendance tracking
5	/api/hr/leave	GET, POST, PUT	Leave management

6	/api/hr/payroll	GET, POST	Payroll processing
7	/api/hr/expenses	GET, POST, PUT	Expense claims
8	/api/hr/performance	GET, POST	Performance reviews
9	/api/hr/recruitment	GET, POST	Job postings and candidates
10	/api/hr/training	GET, POST	Training courses and enrollments
11	/api/hr/chat	POST	AI chatbot endpoint (streaming)
12	/api/hr/dashboard	GET	Dashboard statistics aggregation
13	/api/hr/reports	GET	Report generation and export
14	/api/hr/settings	GET, PUT	System settings management

Table 14: Complete API Endpoints Summary

5.3 Error Handling

All API endpoints return errors in a consistent JSON format. HTTP status codes are used semantically, and error messages are descriptive to aid debugging.

Code	HTTP Status	Description	When It Occurs
VALIDATION_ERROR	400	Request validation failed	Missing/invalid fields
UNAUTHORIZED	401	Authentication required	Missing or invalid token
FORBIDDEN	403	Insufficient permissions	User lacks required role
NOT_FOUND	404	Resource not found	Invalid ID or path
CONFLICT	409	Resource conflict	Duplicate email, active leave
RATE_LIMITED	429	Too many requests	Rate limit exceeded
INTERNAL_ERROR	500	Server error	Unexpected server failure

Table 15: API Error Codes

Error Response Format:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Email address is required",
    "details": [
      { "field": "email", "message": "Required field missing" }
    ]
  }
}
```

5.4 Authentication (Planned)

NextAuth.js integration is planned for the next release. The planned implementation uses JWT-based session management with OAuth2 providers (Google, Microsoft) and credentials-based login. API routes will validate session tokens via NextAuth middleware. Role-based access control will be enforced at the API route level using the Role.permissions JSON field.

5.5 Rate Limiting

Rate limiting is planned using a token bucket algorithm. The proposed limits are: 100 requests per minute for authenticated users, 20 requests per minute for unauthenticated requests, and 10 requests per minute for the AI chat endpoint. Rate limit headers (X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset) will be included in responses.

6. Frontend Architecture

6.1 Component Hierarchy

The frontend follows a hierarchical component structure rooted at the root layout (app/layout.tsx). The layout provides the sidebar navigation and main content area. Each module is rendered as a client component within the main content area. Module components internally use a tab-based layout (Overview, List, Detail, Form) pattern.

Component Hierarchy:

RootLayout → Sidebar + MainContent → ModuleComponent → TabView (Overview | List | Detail | Form)
→ DataTable / FormFields / Charts

6.2 Module Component Structure

Each of the 9 HR modules follows a consistent component pattern. Every module component is a client component ("use client") that receives data from the Zustand store and renders a tabbed interface. The following table documents the module components:

Module	Component	Tabs	Key Features
Dashboard	DashboardModule	Overview	KPI cards, charts, quick actions
Employees	EmployeeModule	Overview, Directory, Detail	CRUD, search, filters, profile
Attendance	AttendanceModule	Overview, Records	Check-in/out, calendar, stats
Leave	LeaveModule	Overview, Requests, Calendar	Apply, approve, balance tracking
Payroll	PayrollModule	Overview, Processing, History	Salary calc, payslips, reports
Expenses	ExpenseModule	Overview, Claims, Approvals	Submit, approve, categorize
Performance	PerformanceModule	Overview, Reviews, Goals	Reviews, ratings, AI insights
Recruitment	RecruitmentModule	Overview, Jobs, Candidates	Post jobs, track pipeline
Training	TrainingModule	Overview, Courses, My Learning	Enroll, track, AI paths

Table 16: Module Components

6.3 State Management with Zustand

The Zustand store is defined in `/src/lib/store.ts`. It uses a single store pattern with typed slices for each module. Actions are defined inline within the store creator. The store persists UI state (active module, sidebar state) to `localStorage` via middleware.

Store Definition Pattern:

```
import { create } from "zustand";
import { persist } from "zustand/middleware";

interface HRStore {
  activeModule: string;
  sidebarCollapsed: boolean;
  setActiveModule: (module: string) => void;
  toggleSidebar: () => void;
}

export const useHRStore = create<HRStore>()(
  persist(
    (set) => ({
      activeModule: "dashboard",
      sidebarCollapsed: false,
      setActiveModule: (module) => set({ activeModule: module }),
      toggleSidebar: () =>
        set((state) => ({ sidebarCollapsed: !state.sidebarCollapsed })),
    }),
    { name: "hr-store" }
  )
);
```

6.4 Data Flow Pattern

The application currently uses mock data defined in `/src/lib/data.ts` for development and demonstration. The migration path to real API integration follows a clear pattern:

- **Current:** Components import mock data directly from `/src/lib/data.ts`
- **Transition:** Replace imports with `fetch()` calls to API routes
- **Future:** Implement React Query (TanStack Query) for caching, revalidation, and optimistic updates

6.5 Routing and Navigation

Navigation is handled through a sidebar component that switches between modules. The active module is stored in the Zustand store and determines which module component is rendered in the main content area. The sidebar supports collapse/expand via the `toggleSidebar` action. On mobile devices, the sidebar becomes a slide-out drawer.

6.6 Responsive Design

The application is built mobile-first using Tailwind CSS responsive utilities. Key breakpoints: sm (640px), md (768px), lg (1024px), xl (1280px). The sidebar automatically collapses on screens below lg. Tables switch to card layout on mobile. Charts are hidden on very small screens with a "view full chart" option.

6.7 Theme System

AI-HRMS uses an emerald/teal professional theme defined in Tailwind CSS configuration. The primary color palette ranges from emerald-50 (#ecfdf5) to emerald-900 (#064e3b), with teal accents for secondary elements. The theme is configured in tailwind.config.ts and extended via CSS custom properties in globals.css. Dark mode support is planned for a future release.

7. Security Architecture

7.1 RBAC Implementation

AI-HRMS implements Role-Based Access Control (RBAC) with a permission matrix stored in the Role.permissions JSON field. Each role defines granular permissions per module, supporting five action types: read, write, modify, delete, and admin. The RBAC system is designed to be enforced at both the frontend (UI element visibility) and API levels.

7.2 Permission Model

Permissions are defined as a matrix where rows represent HR modules and columns represent action types. Each cell contains a boolean indicating whether the role has that permission. The permission check algorithm is: User → Role → permissions[module][action] → allow/deny.

Module	read	write	modify	delete	admin
employees	View employee profiles	Create employees	Edit employee data	Remove employees	Bulk operations
attendance	View attendance	Check in/out	Modify records	Delete records	Manage policies
leave	View own leaves	Submit requests	Approve/reject	Cancel leaves	Manage types
payroll	View own payroll	N/A	Process payroll	N/A	Full access
expenses	View own expenses	Submit claims	Approve claims	Delete claims	Manage categories
performance	View own reviews	Self-assess	Write reviews	Delete reviews	Manage cycles
recruitment	View job postings	Apply for jobs	Manage postings	Delete postings	Full pipeline
training	View courses	Enroll in courses	Manage courses	Delete courses	Create programs
settings	View settings	N/A	N/A	N/A	Full configuration

Table 17: Permission Matrix by Module

7.3 Predefined Roles

The system ships with 7 predefined roles, each with a carefully designed permission set:

Role	Access Level	Key Permissions
Super Admin	Full system access	All modules: admin permission
HR Manager	HR module management	All HR modules: write/modify/admin
HR Staff	HR operations	Employees: write, Leave: modify, Attendance: modify
Department Manager	Department-level management	Own dept employees: modify, Leave: modify, Performance: modify
Employee	Self-service	Own data: read, Leave: write, Attendance: write, Expenses: write
Finance	Payroll and expenses	Payroll: admin, Expenses: admin
Recruiter	Recruitment only	Recruitment: admin, limited employee read

Table 18: Predefined Roles

7.4 Data Masking Strategy

Sensitive data fields are masked based on role permissions. The masking rules are: Salary fields are visible only to the employee themselves, HR Managers, Finance, and Super Admin. Personal contact information (phone, address, emergency contact) is restricted to HR roles and the employee. Performance review details are visible only to the reviewer, the employee, and HR. Masking is applied at the API response level, not just the UI level, to prevent data leakage.

7.5 Audit Logging

The AuditLog model captures all data modification events. Each log entry records: the action performed (CREATE, UPDATE, DELETE), the entity type and ID, the user who performed the action, a JSON diff of changes (before/after values), and a timestamp. Audit logs are immutable and cannot be deleted through the API. Only Super Admin can view audit logs, and they are retained indefinitely for compliance.

7.6 API Security

API security measures include: CSRF protection via SameSite cookies and token validation, input validation using Zod schemas on all request bodies, rate limiting (planned), security headers (X-Content-Type-Options, X-Frame-Options, Content-Security-Policy), and HTTPS enforcement in production. SQL injection is prevented by Prisma parameterized queries. XSS is mitigated by React automatic escaping and Content-Security-Policy headers.

7.7 Input Validation

All API request bodies are validated using Zod schemas before processing. Zod provides runtime type checking with TypeScript type inference. Example schema definition:

```
import { z } from "zod";

export const EmployeeSchema = z.object({
  firstName: z.string().min(1).max(50),
  lastName: z.string().min(1).max(50),
  email: z.string().email(),
  departmentId: z.number().int().positive(),
  roleId: z.number().int().positive(),
  position: z.string().min(1).max(100),
  hireDate: z.string().datetime(),
  salary: z.number().positive().optional(),
});
```

8. AI Integration

8.1 AI Chatbot Implementation

The AI chatbot is implemented using the z-ai-web-dev-sdk and exposed through the /api/hr/chat endpoint. The chatbot provides contextual HR assistance, answering questions about company policies, leave balances, payroll details, and general HR inquiries. Responses are streamed using Server-Sent Events (SSE) for real-time token delivery.

8.1.1 Chat API Endpoint

Endpoint: POST /api/hr/chat

Request Body:

```
{
  "message": "What is the company policy on remote work?",
  "context": {
    "employeeId": "uuid-xxx",
    "module": "dashboard",
    "sessionId": "session-uuid"
  }
}
```

Response: Streaming text/event-signal with token chunks.

8.1.2 System Prompt Engineering

The chatbot uses a carefully engineered system prompt that establishes its role as an HR assistant. The prompt includes: role definition ("You are an AI HR assistant for AI-HRMS"), context injection (current user info, department, role), knowledge boundaries (can only answer HR-related questions), response formatting guidelines (use bullet points for lists, tables for data), and safety guardrails (never reveal other employees' salaries, never provide legal advice).

```
const SYSTEM_PROMPT = `You are an AI HR assistant for AI-HRMS.
You help employees with HR-related queries including:
- Leave balances and policies
- Payroll and compensation questions
- Company policies and procedures
- Training and development resources

Rules:
1. Only answer HR-related questions
2. Never reveal other employees personal data
3. Direct complex issues to HR department
4. Use clear, professional language
5. Format data responses in tables when appropriate`;
```

8.1.3 Conversation History Management

Conversation history is managed client-side in the Zustand store (chatMessages array). Each message includes: role (user/assistant), content (text), timestamp, and sessionId. The last 20 messages are sent with each request for context continuity. Sessions expire after 30 minutes of inactivity. Server-side conversation logging is planned for analytics.

8.2 AI Attrition Prediction

The attrition prediction model analyzes employee data to identify flight risk. The model considers multiple factors weighted by importance: tenure duration, salary competitiveness relative to market, recent performance trends, attendance patterns (increased absences), lack of recent promotions, and training engagement levels. The output is a risk score (0-100) with a risk category (Low/Medium/High/Critical).

Risk Score	Category	Recommended Action
0-25	Low	Monitor via regular reviews
26-50	Medium	Schedule check-in meeting, explore concerns
51-75	High	Immediate retention conversation, counter-offer preparation
76-100	Critical	Escalate to HR leadership, activate retention plan

Table 19: Attrition Risk Categories

8.3 AI Fit Score Calculation

The AI Fit Score evaluates candidate-job compatibility for the recruitment module. The score is calculated as a weighted composite of: skills match (40% weight), experience relevance (25%), culture fit indicators (20%), and growth potential (15%). The score ranges from 0 to 100, with thresholds: Strong Fit (80+), Good Fit (60-79), Potential Fit (40-59), and Weak Fit (below 40). The calculation uses data from the Candidate profile, Job requirements, and historical placement success data.

8.4 AI Learning Path Recommendations

The training module uses AI to recommend personalized learning paths for employees. The recommendation engine considers: current skills vs. required skills for the role, career goals and aspirations, department skill gaps, trending industry skills, and peer learning patterns. Recommendations are surfaced in the Training Module's "My Learning" tab and updated weekly based on new data.

8.5 AI Review Question Generation

During performance review cycles, the AI generates contextual review questions based on: the employee's role and level, department-specific competency frameworks, previous review feedback themes, and current organizational goals. Generated questions are suggestions only; managers can edit, add, or remove questions before sending the review form.

9. Development Workflow

9.1 Local Development Setup

Prerequisites: Node.js 18+, npm or bun, Git

Setup Steps:

```
# 1. Clone the repository
git clone https://github.com/org/ai-hrms.git
cd ai-hrms

# 2. Install dependencies
npm install

# 3. Set up environment variables
cp .env.example .env.local
# Edit .env.local with your configuration

# 4. Set up the database
npx prisma generate
npx prisma migrate dev
npx prisma db seed

# 5. Start the development server
npm run dev
```

9.2 Environment Configuration

Environment variables are managed through `.env.local` (not committed to git). The following variables are required:

Variable	Required	Description	Example
DATABASE_URL	Yes	Prisma database connection string	file:./dev.db
AI_API_KEY	Yes	z-ai-web-dev-sdk API key	sk-xxx...
NEXT_PUBLIC_APP_URL	Yes	Public application URL	http://localhost:3000
NEXTAUTH_SECRET	Planned	NextAuth.js secret key	random-32-char-string
NEXTAUTH_URL	Planned	NextAuth.js callback URL	http://localhost:3000

Table 20: Environment Variables

9.3 Running the Development Server

The development server runs on `http://localhost:3000` by default. Next.js 16 provides fast refresh for instant UI updates. The Prisma Studio visual database browser is available via `npx prisma studio` on port 5555.

9.4 Code Style and Linting

ESLint is configured with Next.js recommended rules and strict TypeScript checks. Prettier handles code formatting with the following key settings: single quotes, 2-space indentation, trailing commas, 80-character print width. Pre-commit hooks via husky and lint-staged ensure code quality before commits.

9.5 Git Workflow

The project follows the Git Flow branching model:

- **main**: Production-ready code, only receives merges from develop and hotfix branches
- **develop**: Integration branch for feature work, auto-deploys to staging
- **feature/***: Individual feature branches, created from develop
- **hotfix/***: Urgent fixes, created from main, merged back to both main and develop
- **release/***: Release preparation branches, created from develop

9.6 Testing Strategy

Test Type	Tool	Scope	CI Integration
Unit Tests	Vitest	Utilities, store actions, helpers	Yes - every PR
Integration Tests	Vitest + MSW	API routes with mocked DB	Yes - every PR
Component Tests	React Testing Library	Component rendering and interaction	Yes - every PR
E2E Tests	Playwright	Critical user flows	Yes - merge to develop
Visual Regression	Playwright screenshots	UI appearance consistency	Planned

Table 21: Testing Strategy

10. Deployment Guide

10.1 Vercel Deployment

Vercel is the recommended deployment platform for AI-HRMS. As the creators of Next.js, Vercel provides zero-configuration deployment with automatic optimizations.

Deployment Steps:

```
# 1. Install Vercel CLI
npm i -g vercel

# 2. Login to Vercel
vercel login

# 3. Link the project
vercel link

# 4. Set environment variables in Vercel dashboard
# or via CLI:
vercel env add DATABASE_URL production
vercel env add AI_API_KEY production

# 5. Deploy
vercel --prod
```

10.2 GitHub Actions CI/CD Pipeline

A GitHub Actions workflow is planned for automated CI/CD. The proposed pipeline:

- **On Pull Request:** Lint check, type check, unit tests, build verification

- **On Merge to Develop:** Deploy to staging environment, run E2E tests
- **On Merge to Main:** Deploy to production, run smoke tests, notify team

Proposed Workflow Configuration:

```
name: CI/CD Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npm run lint
      - run: npm run type-check
      - run: npm run test
      - run: npm run build
```

10.3 Environment Variable Management

Production environment variables are managed through the Vercel dashboard or CLI. Sensitive values (API keys, secrets) should never be committed to the repository. Use Vercel Preview Deployments for testing feature branches with production-like configuration. The `.env.example` file documents all required variables with placeholder values.

10.4 Database Migration in Production

Production database migrations must be applied carefully. The recommended process:

1. Test migration locally against a copy of production data
2. Create a backup of the production database before migration
3. Apply migration using: `npx prisma migrate deploy`
4. Verify migration success in Prisma Studio
5. Monitor application logs for errors post-migration

Important: Never use `prisma migrate dev` or `prisma migrate reset` in production. These commands can cause data loss.

10.5 PostgreSQL Setup for Production

For production, migrate from SQLite to PostgreSQL. The recommended setup uses a managed PostgreSQL service (Vercel Postgres, Supabase, or Neon). Update DATABASE_URL to a PostgreSQL connection string. Prisma handles the dialect differences transparently. Run `npx prisma migrate deploy` to create tables in the new database.

10.6 Monitoring and Logging

Vercel provides built-in monitoring with request logs, function duration metrics, and error tracking. Application-level logging uses structured JSON logs via `console.log` with context objects. Critical errors are logged with full stack traces and request context. Log aggregation via Vercel Log Drains to external services (Datadog, New Relic) is supported.

10.7 Error Tracking

Sentry integration is planned for production error tracking. The implementation will include: automatic exception capture in API routes, React error boundary integration, source map upload for readable stack traces, performance monitoring for slow API routes, and alert rules for critical error thresholds.

11. Performance Optimization

11.1 Rendering Strategy

AI-HRMS uses a hybrid rendering approach leveraging Next.js App Router capabilities:

Strategy	Use Case	Implementation	Benefit
SSR	Dashboard data, employee directory	Server Components	Fresh data on every request
CSR	Interactive modules (charts, forms)	Client Components ("use client")	Rich interactivity, no server round-trip
SSG	Company policies, help docs	<code>generateStaticParams()</code>	Instant load, CDN-cacheable
ISR	Reports, aggregated stats	<code>revalidate = 3600</code>	Periodic refresh without rebuild

Table 22: Rendering Strategies

11.2 API Response Caching

API responses are cached using Next.js built-in caching mechanisms. The caching strategy varies by data freshness requirements: Dashboard stats use revalidation every 5 minutes, employee directory caches for 1 minute, payroll data is not cached (real-time), and static reference data (departments, roles, policies) caches for 1 hour. Cache invalidation is triggered by data mutation (`revalidatePath/revalidateTag` after `POST/PUT/DELETE`).

11.3 Bundle Optimization

Next.js automatically code-splits at the route level. Additional optimizations include: dynamic imports for heavy components (Recharts, rich text editors), tree-shaking unused Radix UI primitives, lazy loading the AI chatbot component, and using next/image for automatic image optimization (WebP, responsive sizes, lazy loading). The target bundle size for the initial page load is under 200KB gzipped.

11.4 Image Optimization

All images use the Next.js Image component which provides: automatic WebP/AVIF conversion, responsive srcset generation, lazy loading with blur placeholder, and CDN caching. Employee avatars are resized to 128x128px at upload time. Large document previews use thumbnail generation with click-to-expand.

11.5 Database Query Optimization

Prisma query performance is optimized through: eager loading with include for related data (avoiding N+1 queries), select for field projection (reducing data transfer), cursor-based pagination for large datasets (instead of offset), and database-level indexes on frequently queried fields. The Prisma query logger is enabled in development to identify slow queries.

N+1 Prevention Example:

```
// BAD: N+1 query (1 query per employee for department)
const employees = await prisma.employee.findMany();
for (const emp of employees) {
  const dept = await prisma.department.findUnique(
    { where: { id: emp.departmentId } }
  );
}

// GOOD: Single query with eager loading
const employees = await prisma.employee.findMany({
  include: { department: true, role: true }
});
```

12. Contributing Guidelines

12.1 Code Style Guide

All contributions must follow the project code style. Key rules:

- TypeScript strict mode is enabled; no any types without explicit justification
- React functional components with explicit return type annotations

- Named exports preferred over default exports
- CSS: Use Tailwind utility classes; avoid custom CSS unless necessary
- Components: One component per file; file name matches component name
- Imports: Group by external, internal, types; alphabetize within groups
- Strings: Single quotes for JavaScript/TypeScript, double quotes in JSX

12.2 PR Process

All changes must go through pull requests. The process is as follows:

- 1. Create a feature branch from develop: `git checkout -b feature/your-feature develop`
- 2. Make changes with clear, atomic commits following the commit convention
- 3. Push and create a PR targeting develop with a descriptive title and description
- 4. Ensure all CI checks pass (lint, type-check, test, build)
- 5. Request review from at least one team member
- 6. Address review feedback with new commits (do not force-push during review)
- 7. Once approved, a maintainer will merge the PR

12.3 Commit Message Conventions

Commit messages follow the Conventional Commits specification:

```
<type>(<scope>): <description>

[optional body]

[optional footer(s)]
```

Type	Description	Example
feat	New feature	feat(attendance): add check-in geolocation
fix	Bug fix	fix(payroll): correct tax calculation formula
docs	Documentation only	docs(api): update employee endpoint docs
style	Code style (no logic change)	style: fix indentation in store.ts
refactor	Code restructuring	refactor(leave): extract approval logic
test	Adding or updating tests	test(employee): add CRUD unit tests
chore	Build, config, tooling	chore: update dependencies

Table 23: Commit Message Types

12.4 Branch Strategy

Branch naming conventions ensure clarity and traceability:

- **Feature branches:** feature/jira-123-short-description
- **Bug fix branches:** fix/jira-456-bug-description
- **Hotfix branches:** hotfix/jira-789-critical-fix
- **Release branches:** release/v1.2.0

Delete branches after merge. Squash merge is preferred for feature branches to maintain a clean history on develop. Merge commits are used for release branches to preserve milestone markers.

13. Troubleshooting

13.1 Common Build Errors

Error	Cause	Solution
Module not found: Can't resolve...	Missing dependency	Run npm install. Clear .next cache with rm -rf .next
Type error: Property does not exist	Stale Prisma client types	Run npx prisma generate to regenerate types
Build timeout on Vercel	Function exceeds 10s build	Check for circular imports; reduce bundle size
Tailwind classes not applying	Purge configuration issue	Verify content paths in tailwind.config.ts
ESLint parse error	TypeScript syntax in JS context	Check eslint config extends next/core-web-vitals

Table 24: Common Build Errors

13.2 Database Connection Issues

Error	Cause	Solution
P1001: Can't reach database server	Invalid DATABASE_URL or server down	Verify connection string; check server status
P2002: Unique constraint failed	Duplicate data on unique field	Check for duplicate email/employeeId before insert
P2025: Record not found	Referenced record doesn't exist	Verify FK references exist before creating relations
P3009: Migration failed	Schema drift from manual DB changes	Reset dev DB: npx prisma migrate reset

SQLite: database is locked	Concurrent write operations	Use WAL mode; switch to PostgreSQL for production
----------------------------	-----------------------------	---

Table 25: Database Connection Issues

13.3 Deployment Issues

Error	Cause	Solution
Build fails on Vercel but works locally	Node.js version mismatch	Set Node.js version in package.json engines field
API routes return 500	Missing environment variables	Verify all required env vars in Vercel dashboard
Static pages show stale data	ISR cache not invalidated	Use revalidatePath() after data mutations
Image optimization fails	Unconfigured image domain	Add domain to next.config.ts images.remotePatterns
Prisma client not generated	Missing postinstall hook	Add "postinstall": "prisma generate" to package.json

Table 26: Deployment Issues

13.4 Performance Issues

Symptom	Likely Cause	Solution
Slow page load (>3s)	Large bundle size	Analyze bundle with next build; use dynamic imports
API response slow (>1s)	N+1 query problem	Enable Prisma query logger; add include for relations
Memory leak in development	Uncleared intervals/listeners	Use useEffect cleanup; check store subscriptions
Dashboard charts laggy	Too many data points	Implement data aggregation; limit chart data points
Hot reload slow	Large component tree	Split large components; use React.memo strategically

Table 27: Performance Issues